hey, i have a small pygame catch game with 4 things to fix/add. doing the first 2 now.

1) collision bug: the basket should only catch an object when it actually falls down to basket height, but right now an object scores the moment it spawns at the top if the basket is under its x, because is_caught in collision.py only checks x and never y. also the catch loop in update() removes from self.objects while looping over it, so it skips objects when 2 are next to each other. fix both.

2) basket control: the basket can go half off the screen on the left and right because the clamp in handle_input doesnt account for the basket width (basket.x is its center). make it always stay fully on screen, and holding an arrow key should stay smooth with no jitter at the edges.

keep everything else the same (screen is 700x500). give me the full collision.py and full game_engine.py, each in its own code block, from the first line to the last, nothing cut off.

Both fixes are done. Only `collision.py` and `game_engine.py` needed changes; `basket.py`, `falling_object.py` and `renderer.py` stay exactly as they were.

Task 1: collision bug

- **Height check.** `is_caught()` now also checks y: the object's bottom edge (`obj.y + obj.radius`) must have reached the basket's top, and its top edge must not be below the basket's bottom. An object that has just spawned can no longer score.
- **List mutation.** The loop no longer calls `remove()` on the list it is iterating. It builds a new `remaining` list and assigns it back, so no object is skipped when two are next to each other.

Task 2: basket control

- `basket.x` is the basket's centre, so the clamp now uses half the basket width: the centre stays between `width / 2` and `WIDTH - width / 2`, which keeps the whole basket on screen.
- Movement still polls the held keys every frame, so holding an arrow key is smooth. Clamping runs every frame, so the basket stops cleanly at an edge instead of jittering.

I ran it headless to check: objects no longer score at the top, two adjacent catches both count, and the basket stops fully at both edges. Want me to move on to the spawning and the speed boost?

> good, that works. now the last 2 things, building on the files you just gave me.
>
> 3) spawning: make objects spawn at varied x positions and varied time gaps instead of a fixed 50 frames. they should never spawn outside the screen, not repeatedly in the same spot, and there should be a cap on how many objects can be on screen at once. put the tuning numbers in constants at the top.
>
> 4) speed boost: when i press space the basket moves faster for about 3 seconds (60 fps) then goes back to normal automatically. show a clear indicator while its active (a countdown text and the basket changes color). space shouldnt restart or stack it while its active. Basket already has unused boosted_frames, normal_speed and boost_speed fields, use those.
>
> important: keep the screen at 700x500 (WIDTH, HEIGHT), keep Basket.__init__(self, x, y, width=90, height=24, speed=5) and its get_rect() as is (basket.y is the center), and keep all existing names in renderer.py (WIDTH, HEIGHT, WINDOW_SIZE, COLOR_BG, COLOR_BASKET, COLOR_TEXT, draw_scene, draw_text, draw_banner). only add to them.
>
> give me the full updated basket.py, game_engine.py and renderer.py, each in its own code block, from the first line to the last, nothing cut off.

Both features are in. I kept the 700x500 screen, the `Basket` constructor and `get_rect()`, and every existing name in `renderer.py`.

Task 3: controlled spawning

- **Varied timing.** The gap to the next spawn is random between `SPAWN_INTERVAL_MIN` and `SPAWN_INTERVAL_MAX` frames (30 to 80).
- **Varied, safe positions.** x is picked between `OBJECT_RADIUS` and `WIDTH - OBJECT_RADIUS`, so objects always spawn fully on screen.
- **No repeats.** A new x must be at least `MIN_SPAWN_DISTANCE` (80 px) from the previous spawn. It retries a few times, then falls back to the far side of the screen.
- **Cap.** `MAX_OBJECTS` limits how many objects can be on screen at once; a spawn is skipped while the cap is reached.

Task 4: speed boost

- **Activation.** Space calls `Basket.activate_boost()`, which sets `boosted_frames` to 180 (3 s at 60 fps) and doubles the speed using the existing `boost_speed` field. It does nothing while a boost is already active, so it can't restart or stack.
- **Expiry.** `Basket.update()` counts the frames down each tick and restores `normal_speed` automatically when it reaches 0.
- **Indicator.** While active, the basket turns yellow and `SPEED BOOST 2.4s` counts down on screen; otherwise a hint shows how to start it.

I ran both stages headless to check them: the basket stops fully at both edges, no more than the cap of objects appears, spawns stay inside the screen, the boost doubles the speed, ignores repeat Space presses, and returns to normal after 180 frames.